

Лабораторный практикум 14. Фролов А.А.

В предыдущих лабораторных работах была разработана программа, выполняющая вывод содержимого сектора памяти на экран. В исходной версии программы управление выполнялось с помощью тестовой процедуры `Imit`, которая последовательно вызывала заранее заданные процедуры. В данной лабораторной работе эта тестовая процедура была заменена на полноценный диспетчер команд.

Для считывания клавиш была добавлена процедура `Read_byte`, использующая прерывание BIOS `int 16h`. Данная процедура позволяет получить не только ASCII-код символа, но и скан-код клавиши. Это необходимо, так как функциональные клавиши не имеют обычного печатного ASCII-кода и различаются именно по скан-кодам. В методическом указании сказано, что после вызова `int 16h` ASCII-код находится в регистре `AL`, а скан-код — в регистре `AH`.

Диспетчер команд реализован с помощью процедур `Dispatcher` и `Command`. Процедура `Dispatcher` выводит приглашение, ожидает ввод клавиши, проверяет нажатие `F10` для завершения программы, а остальные управляющие клавиши передаёт процедуре `Command`.

Процедура `Command` выполняет поиск скан-кода в таблице переходов `Table`. Каждый элемент таблицы состоит из трёх байтов: одного байта скан-кода и двух байтов адреса процедуры-исполнителя. Последний байт таблицы равен `0FFh` и обозначает конец таблицы. Такой способ реализации соответствует методическому указанию, где таблица переходов используется для вызова процедур по скан-кодам функциональных клавиш.

Для команды `F2` была использована процедура `Write_sector`. Она выполняет запись содержимого переменной `Sector` в область памяти, начальный адрес которой хранится в переменной `Address`. В методическом указании отдельно отмечено, что эту процедуру необходимо добавить в файл `Disp_sec.asm`.

В результате программа стала интерактивной и начала реагировать на следующие клавиши:

- `F1` — вывод начального сектора памяти;
- `F2` — запись скорректированного сектора в память;
- `F3` — вывод следующего сектора памяти;
- `F5` — вывод предыдущего сектора памяти;
- `F6` — вывод сектора с заданным номером;
- `F10` — завершение работы программы.

Листинг основных добавленных процедур

Листинг 1 — Процедура чтения клавиши Read_byte

```
1 ; -----
2 ; Ввод символа с клавиатуры без эха
3 ; Выход:
4 ; AL = ASCII-код символа
5 ; AH = скан-код клавиши
6 ; -----
7 Read_byte:
8     mov ah,00h
9     int 16h
10    ret
```

Описание:

Процедура `Read_byte` выполняет ввод одного символа с клавиатуры с помощью BIOS-прерывания `int 16h`. После выполнения процедуры в регистре `AL` находится ASCII-код клавиши, а в регистре `AH` — её скан-код. Это позволяет обрабатывать функциональные клавиши `F1`, `F2`, `F3`, `F5`, `F6` и `F10`.

Листинг 2 — Процедура Dispatcher

```
1 ; -----
2 ; Диспетчер команд
3 ; F1 - вывести начальный сектор
4 ; F2 - записать сектор в память
5 ; F3 - вывести следующий сектор
6 ; F5 - вывести предыдущий сектор
7 ; F6 - вывести сектор N
8 ; F10 - завершить работу программы
9 ; -----
10 Dispatcher:
11     call Clear_screen
12
13 Dispatcher_loop:
14     call Send_crlf
15
16     mov dl,'>'
17     call write_char
18
19     call Read_byte          ; AL = ASCII-код, AH = скан-код
20
21     cmp ah,44h             ; F10?
22     je Dispatcher_exit
23
24     cmp al,00h             ; управляющая клавиша?
25     jne Dispatcher_loop    ; обычные символы игнорируются
26
27     mov dl,ah              ; DL = скан-код команды
```

```

28      call Command
29
30      jmp Dispatcher_loop
31
32  Dispatcher_exit:
33      int 20h

```

Описание:

Процедура `Dispatcher` является главным модулем управления программой. Она очищает экран, выводит приглашение `>`, ожидает нажатия клавиши и анализирует полученный код. Если нажата клавиша `F10`, программа завершает работу. Если нажата управляющая клавиша, её скан-код передаётся в процедуру `Command`.

Листинг 3 — Процедура `Command`

```

1  ; -----
2  ; Интерпретатор команд
3  ; Вход:
4  ; DL = скан-код нажатой функциональной клавиши
5  ; -----
6  Command:
7      push bx
8
9      mov bx,offset Table
10
11  Command_loop:
12      cmp byte ptr [bx],0FFh      ; конец таблицы?
13      je Command_exit
14
15      cmp dl,[bx]                 ; найден скан-код?
16      je Command_dispatch
17
18      add bx,3                    ; переход к следующему элементу
19      jmp Command_loop
20
21  Command_dispatch:
22      inc bx
23      call word ptr [bx]          ; вызов процедуры из таблицы
24
25  Command_exit:
26      pop bx
27      ret

```

Описание:

Процедура `Command` выполняет поиск скан-кода в таблице переходов `Table`. Если скан-код найден, выполняется переход к соответствующей процедуре-

исполнителю. Если код отсутствует в таблице, процедура завершается без выполнения команды.

Листинг 4 — Таблица переходов Table

```
1 ; -----
2 ; Таблица переходов
3 ; 1 байт - скан-код клавиши
4 ; 2 байта - адрес процедуры-исполнителя
5 ; -----
6 Table db 3Bh ; F1
7 dw Init_sector
8
9 db 3Ch ; F2
10 dw Write_sector
11
12 db 3Dh ; F3
13 dw Next_sector
14
15 db 3Fh ; F5
16 dw Prev_sector
17
18 db 40h ; F6
19 dw N_sector
20
21 db 0FFh ; конец таблицы
```

Описание:

Таблица Table содержит соответствие между скан-кодами функциональных клавиш и адресами процедур, которые должны быть выполнены. Благодаря таблице переходов диспетчер легко расширяется: для добавления новой команды достаточно добавить в таблицу новый скан-код и адрес новой процедуры.

Листинг 5 — Процедура Write_sector

```
1 ; -----
2 ; Запись содержимого Sector в сектор памяти
3 ; Адрес сектора хранится в переменной Address
4 ; -----
5 Write_sector:
6     push si
7     push di
8
9     mov si,offset Sector
10    mov di,[Address]
11    call Copy_sector
12
13    pop di
```

```
14     pop si
15     ret
```

Описание:

Процедура `Write_sector` копирует 256 байт из переменной `Sector` в область памяти, адрес которой хранится в переменной `Address`. Для копирования используется ранее разработанная процедура `Copy_sector`.

Компиляция программы:

```
DOSBox-X 2026.01.02: DN - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
14:09:49
Assembling and linking COM file...
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International
Assembling file:  disp_sec.ASM
*Warning* disp_sec.ASM(99) Reserved word used as symbol: TABLE
Error messages:   None
Warning messages: 1
Passes:           1
Remaining memory: 431k
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International
C:\PRAC14>
F1 Help F10 Menu
```

Скомпилированная программа с диспетчером команд:

```
DOSBox-X 2026.01.02: DISP_SEC - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
0100 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 0....0....0....0
0110 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 ....0....0....0.
0120 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 ...0....0....0..
0130 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 ..0....0....0...
0140 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 .0....0....0....
0150 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 0....0....0....0
0160 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 ....0....0....0.
0170 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 ...0....0....0..
0180 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 ..0....0....0...
0190 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 .0....0....0....
01A0 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 0....0....0....0
01B0 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 ....0....0....0.
01C0 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 ...0....0....0..
01D0 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 ..0....0....0...
01E0 01 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 .0....0....0....
01F0 E9 02 01 00 01 E9 02 01 00 01 E9 02 01 00 01 E9 0....0....0....0

> _
```